

Midterm Progress Report

Defect Analysis, Repair, and Python-Port Preparation for the DESTINY Emerging
Memory Modeling Tool

Tiebing Tang 42382277
Yishu Wang 66987866
Zhenyu Liu 41689752

April 13, 2026

1 Project Background

Emerging memory technologies such as STT-RAM, ReRAM, PCRAM, and eDRAM are increasingly important in architecture research. Their latency, dynamic energy, leakage power, and area tradeoffs strongly influence cache and memory hierarchy design. Because silicon prototyping is expensive and slow, early-stage evaluation relies heavily on analytical modeling tools.

DESTINY is a representative academic simulator for SRAM, eDRAM, and several non-volatile memories in both 2D and 3D organizations. It is therefore a valuable baseline for design-space exploration. However, research tools of this kind often accumulate historical assumptions, incomplete features, and mismatches between documentation and implementation. These issues can reduce output credibility and make later extensions more difficult.

The broader goal of this project is to study DESTINY in depth, identify correctness and reproducibility problems, repair high-impact defects, and prepare the codebase for a future Pythonic port. The Python version is intended to make the model easier to maintain, easier to integrate with data-analysis workflows, and easier to extend for new memory technologies.

2 Project Objectives

The project has three main objectives:

1. Understand the internal structure of DESTINY, including configuration parsing, device/cell modeling, array hierarchy modeling, and result aggregation.
2. Identify and repair defects that materially affect correctness, robustness, or reproducibility.
3. Prepare the conceptual foundation for a Python-based reimplementation by clarifying the simulator's data flow and exposing legacy inconsistencies early.

3 Development Environment and Tools

The project is developed and evaluated under the following software environment and toolchain:

- **Operating System:** Ubuntu 22.04.2 LTS running on WSL2
- **Build System:** GNU Make 4.3
- **Version Control:** Git 2.34.1
- **Modeling Framework:** DESTINY (C++ implementation)

The DESTINY simulator used in this project is available at:

- https://github.com/sparsh0mittal/destiny_3d_cache

The project repository can be found at:

- https://github.com/Yuchen-10001/ESE5760_final

4 Work Completed So Far

4.1 Environment Setup

The local build and execution environment has been successfully set up on Windows using MSYS2. The repaired codebase has been compiled with GNU g++ and executed on bundled sample configurations.

The DESTINY framework is compiled using a Makefile provided in the original repository:

```
make
```

4.2 Codebase Exploration

The key modules of the simulator have been inspected and mapped:

- `main.cpp`: top-level flow, technology initialization, search orchestration, and summary output.
- `InputParameter.cpp`: configuration parsing for architectural and technology options.
- `MemCell.cpp`: memory-cell parameter parsing and device-level read/write energy logic.
- `SubArray.cpp`: subarray-level latency and power modeling.
- `Result.cpp`: final reporting and decomposition of latency/energy/area.
- `BankWithHtree.cpp` and `BankWithoutHtree.cpp`: bank-level hierarchy and routing contributions.

This exploration phase established the full path from configuration input to final reported metrics and made it possible to trace suspicious outputs back to their source formulas.

5 Defects Identified

During the audit, several issues with direct impact on correctness or usability were identified.

5.1 Unsupported Process Nodes Caused Undefined Behavior

The original code accepted `ProcessNode` values outside the supported 22 nm to 200 nm range. It then fell through placeholder `TODO` branches and still performed technology interpolation with an invalid or uninitialized `techHigh` state.

Impact: results outside the supported process range were not merely approximate; they were undefined.

5.2 Non-Power-of-Two Associativity Did Not Stop Execution

The simulator printed an error for invalid cache associativity, but the terminating `exit(-1)` was commented out.

Impact: the program could continue with invalid cache geometry assumptions and still produce misleading output.

5.3 The Dedicated eDRAM Technology Path Was Disabled

The code contained conditions of the form:

```
if (cell->memCellType == eDRAM && false)
```

which permanently disabled the intended eDRAM-specific technology initialization.

Impact: eDRAM modeling silently fell back to a more generic peripheral-technology path.

5.4 ReadEnergy Inputs Were Ignored

The bundled `sample_RRAM.cell` file specifies:

```
-ReadEnergy (pJ): 0.1
```

but the parser only recognized `-ReadPower (uW)`.

Impact: a valid user-specified energy input was silently ignored, and the simulator reverted to a derived-power path instead.

5.5 Derived Read Power Could Become Negative

In the current-sensing path for memristor-style cells, the original code computed read current using:

```
(readVoltage - voltageDropAccessDevice) / resistanceOn
```

For the bundled ReRAM sample, `ReadVoltage` = 0.3V and `VoltageDropAccessDevice` = 1.8V, making the effective read voltage negative.

Impact: negative read current propagated into negative read power and eventually into physically impossible negative read-energy terms.

5.6 The SetVoltage Parser Contained a Copy/Paste Bug

The `-SetVoltage` parsing branch reused the `-ResetVoltage` format string and could write to the wrong variable.

Impact: configurations using `SetVoltage` could be parsed incorrectly.

5.7 Documentation and Implementation Used Different Parameter Names

The README used `AllowDifferenceTagTech`, while the parser expected `AllowDifferentTagTech`.

Impact: a user could follow the documentation correctly and still fail to enable the intended feature.

6 Repairs Implemented

At this stage, the highest-impact issues have been repaired directly in the code.

6.1 Added Strict Process-Node Validation

The simulator now terminates immediately if `ProcessNode` is outside the supported 22nm to 200nm range.

Result: invalid technology inputs are now rejected early instead of entering undefined behavior.

6.2 Restored Hard Failure for Invalid Associativity

The commented-out termination logic for non-power-of-two associativity has been restored.

Result: invalid cache configurations no longer produce apparently valid outputs.

6.3 Re-enabled the eDRAM-Specific Technology Path

The hardcoded `&& false` conditions were removed so that eDRAM can use its intended dedicated technology initialization path.

Result: the eDRAM path is now consistent with the original modeling intent.

6.4 Added Support for ReadEnergy

A `readEnergy` field was added to the memory-cell data structure, and the parser now correctly reads:

`-ReadEnergy` (pJ)

Result: explicit read-energy inputs are now honored.

6.5 Corrected Cell Read-Energy Selection Logic

The subarray-level energy path now prefers explicit `readEnergy` when provided. Only if no explicit energy is given does the simulator fall back to a derived read-power path.

Result: user-provided data is no longer silently ignored.

6.6 Prevented Negative Derived Read Power

The read-power estimation function was revised so that:

- it uses the object's own `readMode` field rather than a global pointer,
- the effective read voltage is clamped to a non-negative value,
- explicit `readPower` is returned directly when available.

Result: negative power and negative energy artifacts are eliminated from this path.

6.7 Fixed the SetVoltage Parser

The parser was corrected so that `SetVoltage` is read from the proper line and written to the correct variable.

6.8 Aligned Documentation and Parser Support

The parser now accepts both:

- `AllowDifferentTagTech`
- `AllowDifferenceTagTech`

and the README was updated for consistency.

7 Experimental Validation

To verify that the repairs changed simulator behavior in meaningful ways, several representative runs were performed.

7.1 Validation Case 1: Bundled 3D ReRAM Sample

Command:

```
cd C:\Users\18101\ESE5760_final\config
..\destiny.exe sample_3DReRAM.cfg
```

Before Repair

- Read Dynamic Energy: 150.509 pJ
- Bitline & Cell Read Energy: -0.035 pJ

The negative energy term is physically invalid.

After Repair

- Read Dynamic Energy: 157.047 pJ
- Bitline & Cell Read Energy: 1.600 pJ

Interpretation

- The negative read-energy artifact was removed.
- Total read dynamic energy increased by 6.538 pJ.
- This corresponds to an increase of approximately 4.34%.

This is the strongest quantitative indication so far that the repaired code is more physically consistent than the original baseline.

7.2 Validation Case 2: Bundled STTRAM Cache Sample

Command:

```
cd C:\Users\18101\ESE5760_final\config
..\destiny.exe sample_STTRAM.cfg
```

Observed key results:

- numSolutions = 242209 / numDesigns = 13977846
- Cache Hit Latency: 4.927 ns
- Cache Miss Latency: 3.748 ns
- Cache Write Latency: 6.596 ns
- Cache Hit Dynamic Energy: 0.435 nJ/access
- Cache Miss Dynamic Energy: 0.435 nJ/access
- Cache Write Dynamic Energy: 0.188 nJ/access
- Cache Total Leakage Power: 2538.913 mW

Interpretation: the repair work did not destabilize the main STTRAM cache flow. This is an important regression check because it shows that the changes were targeted rather than disruptive.

7.3 Validation Case 3: Invalid Process Node

A dedicated stress-test configuration was added:

```
config/test_invalid_node.cfg
```

Observed result:

```
[ERROR] Process node 14nm is outside the supported DESTINY  
range of 22nm to 200nm.
```

Interpretation: the simulator now fails fast for unsupported technology inputs.

7.4 Validation Case 4: Invalid Associativity

A second stress-test configuration was added:

```
config/test_invalid_assoc.cfg
```

Observed result:

```
[ERROR] The associativity value has to be a power of 2  
in this version
```

Interpretation: illegal cache geometry inputs are now rejected instead of producing misleading output.

8 Current Project Progress

At the midterm stage, the project has already moved beyond environment setup and code reading. The following concrete progress has been achieved:

1. The simulator can now be compiled and executed locally.
2. The main architecture of the codebase has been analyzed.
3. Several high-impact defects affecting correctness and reproducibility have been identified.
4. A first repair pass has been implemented and validated.
5. At least one repaired issue produced a clear quantitative change in final reported metrics.

This means the project has already transitioned from a purely exploratory phase to an active model-improvement phase.

9 Remaining Limitations

Although the current repair pass addressed several major issues, some known limitations remain:

- Cache miss energy is still treated approximately in the original code.
- The pruning and `doublePrune` paths are still incomplete.
- Some TSV decomposition formulas are still flagged by the original authors as needing revision.
- Bundled sample configurations still depend on execution from the `config` directory due to relative cell-file paths.

These issues are important targets for the next phase.

10 Next-Phase Plan

The next phase of the project will focus on both model reliability and migration readiness.

10.1 Further Simulator Repairs

The next repair targets are:

1. refining cache miss energy so that partial data-array activation is modeled more explicitly,
2. completing or removing unfinished pruning logic,
3. auditing TSV formulas against the original DESTINY publication and technical report,
4. improving path handling for sample configurations.

10.2 Regression Testing

A lightweight regression suite will be added to check:

- that no negative dynamic-energy components appear,
- that invalid inputs fail fast,
- that bundled sample outputs remain stable after changes.

11 Conclusion

At this midterm point, the project has successfully progressed from setup and exploration into concrete simulator improvement. The work completed so far has already produced measurable and reproducible benefits. In particular, the repaired ReRAM sample no longer reports physically impossible negative read energy, and the total read dynamic energy changed by approximately 4.34% after the fix.

These results demonstrate that the project is producing substantive technical progress rather than only descriptive analysis. The current codebase is now in a significantly better state for continued correctness work and for the eventual Pythonic port.

A Sample Output (SRAM 4-Layer Configuration)

The following appendix shows a representative execution output from DESTINY simulator. The cfg file is given by `sample_SRAM_4layer.cfg`.

```
User-defined configuration file (sample_SRAM_4layer.cfg) is loaded
```

```
=====
DESIGN SPECIFICATION
```

```
=====
Design Target: Cache
Capacity      : 2MB
Cache Line Size: 32Bytes
Cache Associativity: 1 Ways
```

```
Searching for the best solution that is optimized for write energy-delay-product ...
Using cell file: sample_SRAM.cell
numSolutions = 413701 / numDesigns = 13977846
```

```
=====
CACHE DESIGN -- SUMMARY
```

```
=====
Access Mode: Normal
Area:
- Total Area = 8.669mm^2
|--- Data Array Area = 3449.472um x 2070.312um = 7.154mm^2
|--- Tag Array Area  = 2170.683um x 697.013um = 1.515mm^2
```

```
Timing:
- Cache Hit Latency   = 0.637ns
- Cache Miss Latency  = 0.316ns
- Cache Write Latency = 0.380ns
```

```
Power:
- Cache Hit Dynamic Energy = 0.354nJ per access
- Cache Miss Dynamic Energy = 0.354nJ per access
```

- Cache Write Dynamic Energy = 0.354nJ per access
- Cache Total Leakage Power = 232.393mW
- |--- Cache Data Array Leakage Power = 183.759mW
- |--- Cache Tag Array Leakage Power = 48.633mW

CACHE DATA ARRAY DETAILS

=====

SUMMARY

=====

Optimized for: Write Energy-Delay-Product
Memory Cell: SRAM
Cell Area (F²) : 146.000 (14.600Fx10.000F)
Cell Aspect Ratio : 1.460
SRAM Cell Access Transistor Width: 1.310F
SRAM Cell NMOS Width: 2.080F
SRAM Cell PMOS Width: 1.230F

=====

CONFIGURATION

=====

Bank Organization: 128 x 32 x 4
- Row Activation : 1 / 128 x 1
- Column Activation: 2 / 32 x 1
Mat Organization: 2 x 2
- Row Activation : 2 / 2
- Column Activation: 2 / 2
- Subarray Size : 8 Rows x 32 Columns
Mux Level:
- Senseamp Mux : 1
- Output Level-1 Mux: 1
- Output Level-2 Mux: 1
- One set is partitioned into 1 rows

Local Wire:

- Wire Type : Local Aggressive
- Repeater Type: No Repeaters
- Low Swing : No

Global Wire:

- Wire Type : Global Aggressive
- Repeater Type: No Repeaters
- Low Swing : No

Buffer Design Style: Latency-Optimized

=====

RESULT

=====

Area:

- Total Area = 3.449mm x 2.070mm = 7.154mm²
- |--- Mat Area = 26.949um x 64.697um = 1743.526um² (144.914%)
- |--- Subarray Area = 13.474um x 22.666um = 305.415um² (206.818%)
- |--- TSV Area = 102425.000um²

```

|--- Logic Layer Area = 77915.487um^2
- Area Efficiency = 144.654%
Timing:
- Read Latency = 502.084ps
|--- TSV Latency      = 0.213ps
|--- H-Tree Latency = 243.721ps
|--- Mat Latency      = 258.149ps
    |--- Predecoder Latency = 127.005ps
    |--- Subarray Latency  = 131.144ps
        |--- Row Decoder Latency = 83.664ps
        |--- Bitline Latency    = 40.725ps
        |--- Senseamp Latency   = 6.755ps
        |--- Mux Latency        = 0.000ps
        |--- Precharge Latency  = 37.665ps
- Write Latency = 380.117ps
|--- TSV Latency      = 0.107ps
|--- H-Tree Latency = 121.861ps
|--- Mat Latency      = 258.149ps
    |--- Predecoder Latency = 127.005ps
    |--- Subarray Latency  = 131.144ps
        |--- Row Decoder Latency = 83.664ps
        |--- Charge Latency    = 0.010ps
- Read Bandwidth  = 375.828GB/s
- Write Bandwidth = 244.006GB/s
Power:
- Read Dynamic Energy = 321.259pJ
|--- TSV Dynamic Energy      = 115.954pJ
|--- H-Tree Dynamic Energy = 198.679pJ
|--- Mat Dynamic Energy      = 6.897pJ per mat
    |--- Predecoder Dynamic Energy = 6.334pJ
    |--- Subarray Dynamic Energy   = 0.141pJ per active subarray
        |--- Row Decoder Dynamic Energy = 0.010pJ
        |--- Mux Decoder Dynamic Energy = 0.022pJ
        |--- Senseamp Dynamic Energy   = 0.041pJ
        |--- Mux Dynamic Energy        = 0.000pJ
        |--- Precharge Dynamic Energy  = 0.054pJ
- Write Dynamic Energy = 320.498pJ
|--- TSV Dynamic Energy      = 108.786pJ
|--- H-Tree Dynamic Energy = 198.679pJ
|--- Mat Dynamic Energy      = 6.517pJ per mat
    |--- Predecoder Dynamic Energy = 6.334pJ
    |--- Subarray Dynamic Energy   = 0.046pJ per active subarray
        |--- Row Decoder Dynamic Energy = 0.010pJ
        |--- Mux Decoder Dynamic Energy = 0.022pJ
        |--- Mux Dynamic Energy        = 0.000pJ
- Leakage Power = 183.759mW
|--- TSV Leakage          = 0.000pW
|--- H-Tree Leakage Power = 0.000pW
|--- Mat Leakage Power    = 11.216uW per mat

```

CACHE TAG ARRAY DETAILS

=====

SUMMARY

=====

Optimized for: Write Energy-Delay-Product
Memory Cell: SRAM
Cell Area (F²) : 146.000 (14.600F x 10.000F)
Cell Aspect Ratio : 1.460
SRAM Cell Access Transistor Width: 1.310F
SRAM Cell NMOS Width: 2.080F
SRAM Cell PMOS Width: 1.230F

=====

CONFIGURATION

=====

Bank Organization: 64 x 64 x 4
- Row Activation : 1 / 64 x 1
- Column Activation: 1 / 64 x 1
Mat Organization: 2 x 1
- Row Activation : 2 / 2
- Column Activation: 1 / 1
- Subarray Size : 4 Rows x 15 Columns

Mux Level:

- Senseamp Mux : 1
- Output Level-1 Mux: 1
- Output Level-2 Mux: 1
- One set is partitioned into 1 rows

Local Wire:

- Wire Type : Local Conservative
- Repeater Type: No Repeaters
- Low Swing : No

Global Wire:

- Wire Type : Global Aggressive
- Repeater Type: No Repeaters
- Low Swing : No

Buffer Design Style: Latency-Optimized

=====

RESULT

=====

Area:

- Total Area = 2.171mm x 697.013um = 1.515mm²
|--- Mat Area = 33.917um x 10.891um = 369.383um² (80.157%)
|--- Subarray Area = 8.667um x 10.891um = 94.390um² (156.843%)
|--- TSV Area = 102425.000um²
|--- Logic Layer Area = 254437.074um²
- Area Efficiency = 80.075%

Timing:

- Read Latency = 315.836ps

```

|--- TSV Latency      = 0.213ps
|--- H-Tree Latency  = 55.867ps
|--- Mat Latency     = 259.755ps
    |--- Predecoder Latency = 79.710ps
    |--- Subarray Latency   = 87.549ps
        |--- Row Decoder Latency = 57.406ps
        |--- Bitline Latency    = 23.386ps
        |--- Senseamp Latency   = 6.757ps
        |--- Mux Latency       = 0.000ps
        |--- Precharge Latency  = 25.289ps
    |--- Comparator Latency = 92.496ps
- Write Latency = 195.299ps
|--- TSV Latency      = 0.107ps
|--- H-Tree Latency  = 27.934ps
|--- Mat Latency     = 167.259ps
    |--- Predecoder Latency = 79.710ps
    |--- Subarray Latency   = 87.549ps
        |--- Row Decoder Latency = 57.406ps
        |--- Charge Latency    = 0.004ps
- Read Bandwidth  = 67.651GB/s
- Write Bandwidth = 42.833GB/s
Power:
- Read Dynamic Energy = 33.096pJ
|--- TSV Dynamic Energy = 18.974pJ
|--- H-Tree Dynamic Energy = 14.744pJ
|--- Mat Dynamic Energy = 4.859pJ per mat
    |--- Predecoder Dynamic Energy = 4.742pJ
    |--- Subarray Dynamic Energy = 0.059pJ per active subarray
        |--- Row Decoder Dynamic Energy = 0.005pJ
        |--- Mux Decoder Dynamic Energy = 0.010pJ
        |--- Senseamp Dynamic Energy = 0.019pJ
        |--- Mux Dynamic Energy = 0.000pJ
        |--- Precharge Dynamic Energy = 0.021pJ
- Write Dynamic Energy = 33.015pJ
|--- TSV Dynamic Energy = 13.493pJ
|--- H-Tree Dynamic Energy = 14.744pJ
|--- Mat Dynamic Energy = 4.778pJ per mat
    |--- Predecoder Dynamic Energy = 4.742pJ
    |--- Subarray Dynamic Energy = 0.018pJ per active subarray
        |--- Row Decoder Dynamic Energy = 0.005pJ
        |--- Mux Decoder Dynamic Energy = 0.010pJ
        |--- Mux Dynamic Energy = 0.000pJ
- Leakage Power = 48.633mW
|--- TSV Leakage = 0.000pW
|--- H-Tree Leakage Power = 0.000pW
|--- Mat Leakage Power = 2.968uW per mat

```

Finished!